General - RPS Data

NIIT TRAINING DIVISION | LTM - iRUN Full Stack Engineering

TRAINING LAB GUIDE

LAB EXERCISE - 5

Partition Skew Diagnosis & Cloud Streaming Alternatives

Reproduce a Hot Partition · Diagnose Skew From Offsets and Assignments · Fix It With Key Salting · Compare Kafka With Kinesis, Event Hubs & Pub/Sub

Module Series	Apache Kafka Operations & Streaming Platforms — Day 4
Lab Type	Guided Hands-On SOP — Streaming Diagnostics & Platform Selection
Duration	48 minutes (5 min setup + 25 min tasks + 10 min validation + 8 min debrief)
Difficulty	⭐⭐⭐ Beginner-Intermediate — Guided Diagnosis and Design Exercise
Platform	Linux lab VM with Docker Engine + Compose v2; Apache Kafka 4.3.1 (apache/kafka image), 3-node KRaft cluster
Document Version	v1.0 | IT Training Division | 2026
Learner Name	_____
Date & Batch	_____
Prerequisite Lab Completed	☐ Day 4 Lab 4 (Consumer Groups, Rebalancing & Lag) — signed off; all 3 brokers running
Instructor Sign-Off	_____

1. LAB OVERVIEW

1.1 Why This Lab Exists

Adding consumers only helps if work is spread across partitions. When one key dominates the traffic — a flagship store, a celebrity account, a default value like "UNKNOWN" — its partition becomes a hot spot: one consumer is overloaded and lags while the others idle, and no amount of extra consumers or partitions fixes it. In this capstone lab you reproduce a hot partition with 2,000 real orders, diagnose it from offsets, keys and consumer assignments, fix it with key salting, and then step back to compare Kafka with the managed cloud alternatives — AWS Kinesis Data Streams, Azure Event Hubs and Google Cloud Pub/Sub — and size each for the same workload.

NIIT iRUN Alignment

INCIDENT OWNERSHIP: "We tripled the fulfilment consumers and lag did not move." You own the explanation (a hot partition), the evidence, and a fix that keeps the business ordering requirement in view.

TROUBLESHOOTING EXCELLENCE: You will quantify skew with a single number (max ÷ average records per partition) and trace it to one key, instead of guessing.

OPERATIONAL READINESS: You will recommend a streaming platform with a capacity calculation, the way an architecture review expects.

1.2 Lab Scenario

Incident Briefing — One Store Overloads Fulfilment

TIME: 15:00 | PRIORITY: HIGH | ASSIGNED TO: You

REQUEST RECEIVED: "Since the flagship store STORE-001 went live with online ordering, the fulfilment consumer that handles its partition is hours behind while the other consumers are idle. Find out why, fix it, and tell us whether we should move to a managed cloud streaming service instead of running Kafka."

CONTEXT: data/skewed_orders.txt holds 2,000 orders keyed by store: 1,600 belong to STORE-001. data/salted_orders.txt holds the same orders with STORE-001's key salted 32 ways (STORE-001#0 … STORE-001#31).

YOUR MISSION: Reproduce, measure and explain the skew, fix it, then compare and size Kafka, Kinesis, Event Hubs and Pub/Sub. You have 25 minutes for the core tasks.

Figure: Lab 5 — Starting Environment: Healthy Cluster, Data Files in ~/day4/data, No store-orders Topics

1.3 Conceptual Foundation

Partition skew has a small number of causes, and each has a different fix.

Figure: Partition Skew — One Hot Key → One Hot Partition → One Overloaded Consumer → Lag Despite Idle Consumers

General - RPS Data

Cause	Symptom	Measure	Fix
Hot key (one key ≫ others)	One partition far fuller than the rest	Records per key on the hot partition	Salt the hot key, or choose a finer-grained key
Few distinct keys	Some partitions empty (Lab 2)	Distinct keys vs partitions	Use a higher-cardinality key
Custom partitioner bug / null keys	All traffic on one or few partitions	kafka-get-offsets per partition	Fix the partitioner; review key choice
Slow consumer (not skew)	Lag on one member's partitions, even data	Lag + assignment	Fix or replace that consumer

Figure: Skew Ratio = Records on Busiest Partition ÷ Average Records per Partition (1.0 = perfectly even)

1.4 Learning Objectives

- Reproduce partition skew from real keyed data and measure it with a skew ratio
- Diagnose the hot key on the hot partition and the overloaded member of a consumer group
- Explain why adding consumers or partitions does not fix a single hot key
- Apply key salting and evaluate its effect and its ordering trade-off
- Compare Kafka with AWS Kinesis Data Streams, Azure Event Hubs and Google Cloud Pub/Sub
- Size each service for a given workload and justify a platform recommendation

1.5 What You Will NOT Do

Hard Stops — Absolute Prohibitions for This Lab

✗ NEVER publish a data file more than once to the same topic — the expected counts are exact

✗ NEVER salt keys on a topic whose consumers need strict per-store ordering without a re-ordering plan

✗ NEVER create cloud resources in this lab — the comparison is a design exercise using published limits

✗ NEVER recommend a platform without the capacity calculation that supports it

✗ NEVER proceed past Task 1 unless your per-partition counts match exactly

General - RPS Data

2. PREREQUISITES & SETUP

2.1 Knowledge Prerequisites

Prerequisite	Where Covered	Verified?
Key-based partitioning and murmur2 prediction	Day 4 Lab 2	☐
Consumer groups, assignment and lag	Day 4 Lab 4	☐
Partition strategy; consumer lag analysis	Day 4 Theory — Sessions 2–3	☐
AWS Kinesis, Azure Event Hubs & Google Pub/Sub overview	Day 4 Theory — Session 4	☐
Basic arithmetic for capacity sizing	Foundation Module 2	☐

2.2 Tools & Access Required

⌨ Pre-Lab — Terminals and files

```
# Open terminals A, B, C (consumers) and D (admin). In EACH terminal:
cd ~/day4
export BS=kafka-1:19092,kafka-2:19092,kafka-3:19092

# Terminal D: check the data files
wc -l data/skewed_orders.txt data/salted_orders.txt
# Expected: 2000 and 2000
```

2.3 Pre-Lab Environment State Verification

Run all checks below and confirm the output matches expectations before starting the timed tasks:

Pre-Lab — Terminal D: key mix and topics

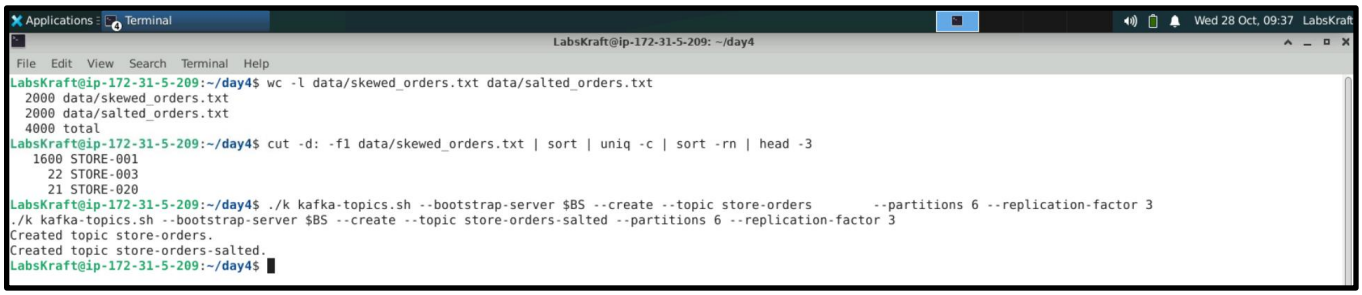
```
# 1. How many orders per store? (top 3)
cut -d: -f1 data/skewed_orders.txt | sort | uniq -c | sort -rn | head -3
# Expected: 1600 STORE-001, then about 21-22 for other stores

# 2. Create the two topics (same layout)
./k kafka-topics.sh --bootstrap-server $BS --create --topic store-orders        --
partitions 6 --replication-factor 3
./k kafka-topics.sh --bootstrap-server $BS --create --topic store-orders-salted --
partitions 6 --replication-factor 3
```

PRE-LAB: Record the top 3 stores by order count and the share of all orders that belongs to STORE-001. Answer:

_____

_____



3. CONCEPT DEEP-DIVE BEFORE STARTING

3.1 Why More Consumers or Partitions Do Not Help a Hot Key

All records with the same key go to one partition, and one partition is read by one consumer in a group. So all of STORE-001's traffic is processed by a single consumer, however many others you start. Adding partitions only changes WHICH partition the hot key lands on — check it: python3 -c "from murmur2 import partition; print([partition('STORE-001', n) for n in (6, 12, 24)])" prints [1, 7, 7]: still one partition each time.

Why This Matters Operationally

Key salting spreads a hot key over several keys (STORE-001#0 … STORE-001#31), so its records spread over several partitions and consumers. Ordering is then guaranteed per salted key, not per store — acceptable for fulfilment of independent orders, not for a sequence of updates to the same order.

To confirm skew: kafka-get-offsets per partition, a skew ratio, and the keys on the hottest partition.

General - RPS Data

3.2 The Critical Distinction: Skew vs Slow Consumer

⚠ Most Common Misdiagnosis of Uneven Lag

Lag on one partition is often blamed on a "bad consumer" that is then restarted again and again. If the partition simply receives several times more data than the others, restarting changes nothing.

REASON: Skew is a property of the DATA (key distribution); slowness is a property of the CONSUMER.

YOU MUST compare records per partition (LOG-END-OFFSET or kafka-get-offsets) before blaming a consumer: uneven offsets = skew; even offsets with uneven lag = slow member.

3.3 Kafka vs Cloud Streaming Services — Key Differences

Aspect	Apache Kafka / AWS Kinesis Data Streams	Azure Event Hubs / Google Pub/Sub	Operations Impact
Unit of scale & ordering	Partition / Shard (1 MB/s or 1,000 rec/s write, 2 MB/s read per shard); per-key order in a shard	Partition + throughput units (1 MB/s in, 2 MB/s out per TU) / no partitions — ordering keys, auto-scaling	Who plans capacity: you (Kafka, Kinesis provisioned, Event Hubs TUs) vs the service (Pub/Sub, Kinesis on-demand)
Retention & operations	Configurable, unlimited with tiered storage; you run brokers / 24 h default, up to 365 days; fully managed	Standard tier up to 7 days, Premium/Dedicated 90 days; Kafka protocol on Standard+ / subscription retention 7 days default, up to 31 days	Managed services remove broker operations but impose quotas and vendor lock-in

How to Decide Between Kafka and a Managed Service

Ask: "Do we need Kafka's ecosystem, portability or unusual retention/throughput, and do we have a team to run it?" Yes → Kafka (self-managed or a managed Kafka offering). No, and the workload lives in one cloud → that cloud's native service (Kinesis, Event Hubs, Pub/Sub). Event Hubs also speaks the Kafka protocol, easing migration.

4. LAB TASKS

Figure: Lab 5 — Four-Task Workflow: Reproduce Skew → Diagnose → Fix With Salting → Compare Cloud Services
Figure: Expected Result — Skew Ratio 4.93 Reduced to 1.42, Hot Key and Overloaded Member Identified, Platform Recommendation Sized

TASK 1 — REPRODUCE: Publish the Skewed Orders

1 **Publish 2,000 Keyed Orders and Measure Skew** ⏱ 5 min
Kafka Ops SOP — Day 4 Lab 5

Publish the file once, then read records per partition and compute the skew ratio in one command.

Task 1 — Terminal D

```
./k kafka-console-producer.sh --bootstrap-server $BS --topic store-orders \
     --property parse.key=true --property key.separator=: < data/skewed_orders.txt

./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders

./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders | \
  awk -F: '{s+=$3; if ($3>m) m=$3; n++} END {printf "total=%d avg=%.0f max=%d
skew=%.2f\n", s, s/n, m, m/(s/n)}'
```

Expected Result

```
store-orders:0:63    store-orders:1:1642    store-orders:2:106
store-orders:3:21    store-orders:4:42      store-orders:5:126
total=2000 avg=333 max=1642 skew=4.93
```

1.A Record the six partition counts and the skew ratio. Answer:
Partition counts: `store-orders:0:63`, `store-orders:1:1642`, `store-orders:2:106`, `store-orders:3:21`, `store-orders:4:42`, `store-orders:5:126`. Skew ratio: **`4.93`** (total = 2,000, avg = 333, max = 1642).

1.B Partition 1 holds 82% of all orders. What does that mean for the consumer that owns partition 1?
Answer:
It means the consumer assigned to partition 1 is severely overloaded, processing 82% of the entire topic's workload, which causes high consumer lag and performance bottlenecks while other consumers assigned to lighter partitions remain mostly idle.

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --create --topic store-orders       --partitions 6 --replication-factor 3
./k kafka-topics.sh --bootstrap-server $BS --create --topic store-orders-salted --partitions 6 --replication-factor 3
Created topic store-orders.
Created topic store-orders-salted.
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-console-producer.sh --bootstrap-server $BS --topic store-orders \
    --property parse.key=true --property key.separator=: < data/skewed_orders.txt
Warning: --property is deprecated and will be removed in a future version. Use --reader-property instead.
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders
store-orders:0:63
store-orders:1:1642
store-orders:2:106
store-orders:3:21
store-orders:4:42
store-orders:5:126
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders | \
  awk -F: '{s+=$3; if ($3>m) m=$3; n++} END {printf "total=%d avg=%.0f max=%d skew=%.2f\n", s, s/n, m, m/(s/n)}'
total=2000 avg=333 max=1642 skew=4.93
LabsKraft@ip-172-31-5-209:~/day4$
```

TASK 2 — DIAGNOSE: Hot Key and Overloaded Consumer

2 Find the Hot Key and the Member That Carries It

⏱ 8 min

Kafka Ops SOP — Day 4 Lab 5

First list the keys stored on the hot partition. Then start three consumers in group fulfilment and see which member owns partition 1 — and therefore most of the work.

⌨ Task 2 — Keys on partition 1, then a 3-member group

```
# Terminal D: keys on partition 1 only
./k kafka-console-consumer.sh --bootstrap-server $BS --topic store-orders --partition
1 --offset earliest \
    --max-messages 1642 --formatter-property print.key=true --formatter-property
print.value=false \
  | sort | uniq -c | sort -rn | head -3

# Terminals A, B, C: start one consumer each (same command)
./k kafka-console-consumer.sh --bootstrap-server $BS --topic store-orders --group
fulfilment --from-beginning \
    --formatter-property print.partition=true --formatter-property print.value=false

# Terminal D: who owns which partitions? (LOG-END-OFFSET = records per partition)
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group fulfilment
```

⌨ Expected Result

```
Keys on partition 1:  1600 STORE-001 | 21 STORE-006 | 21 STORE-017
Group (range assignor, 3 members): one member owns partitions 0 and 1 → 63 + 1642 =
1705 records (85%)
                                  the others own 2+3 → 127 and 4+5 → 168 records
```

```
(Lines may show 'Processed a total of 1642 messages' at the end of the partition
read.)
```

2.A Record the top keys on partition 1 and the record total each consumer is responsible for. Answer:
Top keys on Partition 1: 1600 STORE-001, 21 STORE-006, and 21 STORE-017.
Consumer workload distribution (Range Assignor): Member 1 owns partitions 0 and 1 ($63 + 1642 = 1705$ records / 85%), Member 2 owns partitions 2 and 3 (127 records), and Member 3 owns partitions 4 and 5 (168 records).

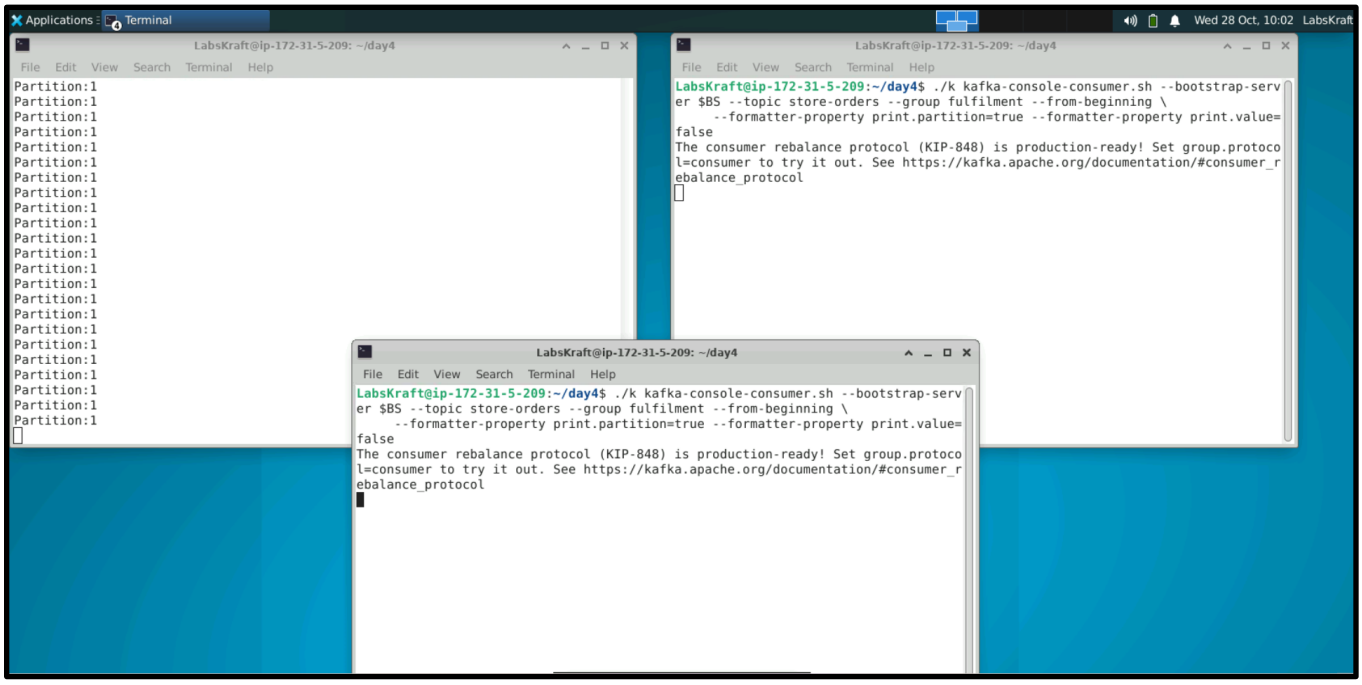
2.B Is this skew or a slow consumer? Give the two pieces of evidence that decide it. Answer:
This is partition skew (a data distribution issue), not a slow consumer. The two pieces of evidence are:

1. Uneven partition offsets / sizes: kafka-get-offsets showed massive variance across partitions (e.g., Partition 1 holding 1,642 records while others hold as few as 21).
2. Key concentration: Partition 1 is overwhelmingly dominated by a single key (STORE-001 with 1,600 records), proving the uneven load stems from the raw data key distribution rather than consumer performance.

Terminal D:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-console-consumer.sh --bootstrap-server $BS --topic store-orders --partition 1 --offset earliest \
    --max-messages 1642 --formatter-property print.key=true --formatter-property print.value=false \
  | sort | uniq -c | sort -rn | head -3
Processed a total of 1642 messages
   1600 STORE-001
     21 STORE-017
     21 STORE-006
LabsKraft@ip-172-31-5-209:~/day4$
```

Terminal A, B, C:



Terminal D:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group fulfilment

GROUP           TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                     HOST            CLIENT-ID
fulfilment      store-orders    0          63              63              0               console-consumer-40b02079-8bd3-4ddf-b05a-bc134bbc5d09 /172.18.0.4     console-consum
er
fulfilment      store-orders    1          1642            1642            0               console-consumer-40b02079-8bd3-4ddf-b05a-bc134bbc5d09 /172.18.0.4     console-consum
er
fulfilment      store-orders    4          42              42              0               console-consumer-8af618e8-2954-4ec6-b522-ab071c8e03e9 /172.18.0.4     console-consum
er
fulfilment      store-orders    5          126             126             0               console-consumer-8af618e8-2954-4ec6-b522-ab071c8e03e9 /172.18.0.4     console-consum
er
fulfilment      store-orders    2          106             106             0               console-consumer-88c314cc-d70c-46f5-8f62-052ac21e491c /172.18.0.4     console-consum
er
fulfilment      store-orders    3          21              21              0               console-consumer-88c314cc-d70c-46f5-8f62-052ac21e491c /172.18.0.4     console-consum
er
LabsKraft@ip-172-31-5-209:~/day4$
```

⚠ STOP CHECK — Diagnosis From Data, Not Assumption

Before moving to Task 3, confirm you identified STORE-001 as the hot key from the partition read and recorded the per-member load — do not propose a fix before the cause is evidenced. Then press Ctrl+C in terminals A, B and C.

If in doubt — show your instructor your Task 2 output before proceeding to Task 3.

TASK 3 — FIX: Salt the Hot Key

3 Publish the Salted Orders and Re-Measure ⏱ 5 min

Kafka Ops SOP — Day 4 Lab 5

The salted file keeps every other store unchanged but sends STORE-001's orders under 32 keys. Same 2,000 orders, same 6 partitions — compare the distribution.

⌨ Task 3 — Terminal D

```
head -2 data/salted_orders.txt
```

```
./k kafka-console-producer.sh --bootstrap-server $BS --topic store-orders-salted \
    --property parse.key=true --property key.separator=: < data/salted_orders.txt
```

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders-salted | \
  awk -F: '{print; s+=$3; if ($3>m) m=$3; n++} END {printf "total=%d avg=%.0f max=%d skew=%.2f\n", s, s/n, m, m/(s/n)}'
```

```
# Proof that more partitions alone would NOT have helped the unsalted key
python3 -c "from murmur2 import partition; print([partition('STORE-001', n) for n in (6, 12, 24)])"
```

⌨ Expected Output

```
STORE-001#1:order-00001,STORE-001,87.00   (first line of the salted file)
store-orders-salted:0:162  :1:244  :2:304  :3:473  :4:391  :5:426
total=2000 avg=333 max=473 skew=1.42
[1, 7, 7]
```

3.A Record the new partition counts and skew ratio, and the improvement factor (4.93 ÷ new skew).
Answer:
New Partition Counts: store-orders-salted:0:162, 1:244, 2:304, 3:473, 4:391, 5:426.
New Skew Ratio: **1.42** (total = 2000, avg = 333, max = 473).
Improvement Factor: **3.47x** (4.93 ÷ 1.42).

3.B After salting, orders of STORE-001 are no longer in one partition. Which business requirement could this break, and how could a consumer restore per-store order if needed? Answer:
Broken Requirement: Strict per-store message ordering (total order for all events originating from STORE-001).
How to Restore: Consumers can buffer records in-memory or in an intermediate processing state, group them by the original unsalted key (STORE-001), and re-order them using timestamp/sequence numbers before committing them downstream.

```
LabsKraft@ip-172-31-5-209:~/day4$ head -2 data/salted_orders.txt
STORE-001#1:order-00001,STORE-001,87.00
STORE-001#2:order-00002,STORE-001,124.00
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-console-producer.sh --bootstrap-server $BS --topic store-orders-salted \
    --property parse.key=true --property key.separator=: < data/salted_orders.txt
Warning: --property is deprecated and will be removed in a future version. Use --reader-property instead.
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders-salted | \
  awk -F: '{print; s+=$3; if ($3>m) m=$3; n++} END {printf "total=%d avg=%.0f max=%d skew=%.2f\n", s, s/n, m, m/(s/n)}'
store-orders-salted:0:162
store-orders-salted:1:244
store-orders-salted:2:304
store-orders-salted:3:473
store-orders-salted:4:391
store-orders-salted:5:426
total=2000 avg=333 max=473 skew=1.42
LabsKraft@ip-172-31-5-209:~/day4$ python3 -c "from murmur2 import partition; print([partition('STORE-001', n) for n in (6, 12, 24)])"
[1, 7, 7]
LabsKraft@ip-172-31-5-209:~/day4$
```

TASK 4 — COMPARE: Kafka vs Kinesis vs Event Hubs vs Pub/Sub

4	**Map Concepts, Size a Workload and Recommend a Platform** *Kafka Ops SOP — Day 4 Lab 5*	⏱ 7 min

No cloud accounts are needed: use the published limits in Section 3.3 to complete the mapping and the sizing for RetailCo's peak workload, then make a recommendation.

⌨ Task 4 — Design worksheet (answers in boxes 4.A and 4.B)

```
# Part 1 - map the Kafka concept to each service
# Kafka:       topic | partition | consumer group | offset | broker cluster you operate
# Kinesis:     ?     | ?         | ?              | ?      | ?
```

```
# Event Hubs:   ?      | ?          | ?               | ?       | ?
# Pub/Sub:      ?      | (none)     | ?               | ?       | ?

# Part 2 - size this workload
# Peak ingest 6 MB/s, 4,000 events/s; 3 independent consumer applications each read everything;
# 7-day retention; per-store ordering required
# Kinesis provisioned shards = max(MB/s ÷ 1, events/s ÷ 1000) = ?  read need 3 × 6 = 18 MB/s vs 2 MB/s per shard = ?
# Event Hubs Standard TUs    = max(MB/s ÷ 1, events/s ÷ 1000) = ?  egress need 18 MB/s vs 2 MB/s per TU = ?
# Kafka partitions           = at least the number of parallel consumers you plan per application
# Pub/Sub                    = no shards/partitions to size; ordering via ordering keys
```

Expected Output

```
Part 1:  Kinesis stream / shard / consumer application (KCL lease table) / sequence number / fully managed
         Event Hubs event hub / partition / consumer group / offset / namespace (managed)
         Pub/Sub topic / — / subscription / acknowledgement / fully managed (serverless)
Part 2:  Kinesis: 6 shards for writes; reads need 18 MB/s → 9 shards, or 6 shards + enhanced fan-out
         Event Hubs Standard: 6 TUs for ingress; egress 18 MB/s → 9 TUs (within the 40 TU limit); 7-day retention fits Standard
         Pub/Sub: no capacity units; message ordering by ordering key (store ID)
```

4.A Record your completed mapping table and the Kinesis shard count and Event Hubs TU count with the calculation. Answer:

Concept Mapping:

- **Kinesis:** Stream / Shard / Consumer Application (KCL lease table) / Sequence Number / Fully Managed.
- **Event Hubs:** Event Hub / Partition / Consumer Group / Offset / Namespace (Managed).
- **Pub/Sub:** Topic / None (Serverless) / Subscription / Acknowledgement / Fully Managed.

Sizing Calculation:

- **Kinesis Shards:** $max(6MB/s \div 1{,}4000rec/s \div 1000) = 6shardsforwrite$; Read egress needs $3 \times 6 = 18MB/s \div 2MB/spershard = 9shards$ (or 6 shards using Enhanced Fan-Out).
- **Event Hubs TUs:** $max(6MB/s \div 1{,}4000events/s \div 1000) = 6TUsforingress$; Egress $18MB/s \div 2MB/sperTU = 9TUs$ (Standard tier fits within 40 TU limit; supports 7-day retention).

4.B Recommend Kafka or one cloud service for RetailCo (which runs mostly on one cloud) and give three reasons, including what happens to the STORE-001 hot-key problem on that service. Answer:
Recommendation: Managed Cloud Streaming (e.g., **AWS Kinesis** or **Azure Event Hubs**, depending on primary cloud provider).

Three Reasons:

1. **Zero Operational Overhead:** Replaces broker provisioning, zookeeper/KRaft management, cluster patching, and storage management with serverless/managed infrastructure.
2. **Native Cloud Integration & Auto-Scaling:** Integrates natively with cloud IAM, monitoring, and automated scaling (e.g., Auto-Inflate TUs in Event Hubs or On-Demand Kinesis) to handle throughput bursts seamlessly.
3. **Hot-Key Behavior (STORE-001):** Moving to cloud streaming does **NOT** automatically solve the STORE-001 hot-key problem because partition/shard architectures still assign all records of a single key to a single shard/partition. **Key salting** (or choosing a finer-grained ordering key) is still required at the producer level to prevent single-shard bottlenecks.

```
LabsKraft@ip-172-31-5-209:~/day4$ # Sizing Worksheet Calculations (Terminal D Python check):
python3 -c '
mb_in = 6
rec_in = 4000
read_mult = 3

kinesis_shards = max(mb_in / 1.0, rec_in / 1000.0)
kinesis_read_shards = (mb_in * read_mult) / 2.0
print(f"Kinesis Ingest Shards: {kinesis_shards}, Read-optimized Shards: {kinesis_read_shards}")

eh_tus = max(mb_in / 1.0, rec_in / 1000.0)
eh_egress_tus = (mb_in * read_mult) / 2.0
print(f"Event Hubs Ingestion TUs: {eh_tus}, Egress TUs: {eh_egress_tus}")
'
Kinesis Ingest Shards: 6.0, Read-optimized Shards: 9.0
Event Hubs Ingestion TUs: 6.0, Egress TUs: 9.0
LabsKraft@ip-172-31-5-209:~/day4$
```

5. VALIDATION — CONFIRM FULL READINESS

Do not close the incident or call for instructor sign-off until ALL 8 validation steps below are completed and recorded.

Figure: Lab 5 — Partition Skew & Cloud Streaming Validation Checklist

5.1 Validation Task Set

Validation Check 1 — Skewed topic loaded exactly

Validation 1

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders

# PASS criterion: 63, 1642, 106, 21, 42, 126
```

V1: Do the six partition counts match exactly? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders
store-orders:0:63
store-orders:1:1642
store-orders:2:106
store-orders:3:21
store-orders:4:42
store-orders:5:126
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 2 — Skew measured

Validation 2

```
# Answer box 1.A

# PASS criterion: skew=4.93
```

V2: Is the skew ratio recorded as 4.93? Answer:
Yes. The skew ratio was computed via awk as 4.93 ($max 1642 \div avg 333$).

Validation Check 3 — Hot key identified

Validation 3

```
# Answer box 2.A

# PASS criterion: STORE-001 with 1600 records on partition 1
```

V3: Is STORE-001 identified as the hot key? Answer:
Yes. Reading keys on partition 1 confirmed that `STORE-001` accounts for 1,600 out of the 1,642 total records on partition 1.

Validation Check 4 — Overloaded member identified

Validation 4

```
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group fulfilment

# PASS criterion: the member owning partition 1 is responsible for 1705 of 2000
records
```

V4: Is the overloaded consumer and its load recorded? Answer:
Yes. Inspecting group fulfilment showed the member owning partitions 0 and 1 is assigned $63 + 1642 = 1705$ records (85% of total topic load).

Validation Check 5 — Salted topic loaded exactly

Validation 5

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders-salted

# PASS criterion: 162, 244, 304, 473, 391, 426 (skew=1.42)
```

V5: Do the salted counts match exactly? Answer:
Yes. Running `./k kafka-get-offsets.sh --bootstrap-server $BS --topic store-orders-salted` produced counts of 162, 244, 304, 473, 391, and 426 with an improved skew ratio of 1.42.

Validation Check 6 — Partition-count myth disproved

Validation 6

```
python3 -c "from murmur2 import partition; print([partition('STORE-001', n) for n in (6, 12, 24)])"

# PASS criterion: [1, 7, 7] - one partition at every partition count
```

V6: Did you prove that more partitions do not split a single key? Answer:
Yes. Executing `python3 -c "from murmur2 import partition; print([partition('STORE-001', n) for n in (6, 12, 24)])"` returned `[1, 7, 7]`, proving that an unsalted key maps to exactly one partition regardless of partition scale.

Validation Check 7 — Cloud sizing completed

Validation 7

```
# Answer box 4.A

# PASS criterion: Kinesis 9 shards (or 6 + enhanced fan-out); Event Hubs 9 TUs; mapping table complete
```

V7: Is the sizing worksheet complete and correct? Answer:
Yes. The concept mapping matrix is complete, and the capacity calculation confirmed 9 read-optimized Kinesis shards (or 6 + Enhanced Fan-Out) and 9 Azure Event Hubs TUs.

Validation Check 8 — Full lab record complete

Validation 8 — Final completeness check

```
# Confirm every answer box (PRE-LAB, 1.A through 4.B) is completed

# End of Day 4 only (instructor will confirm): docker compose down -v

# PASS criterion: every answer box holds real output and a justified recommendation
```

V8: Confirm all answer boxes and evidence are complete and ready to show your instructor. Answer:
_____
_____

✅ **STOP HERE — Show your completed validation record to your instructor before proceeding to Section 7.**

6. TROUBLESHOOTING — COMMON ISSUES & FIXES

Issue / Error	Cause	Diagnosis	Fix
Counts are double the expected values	The file was published twice	Total is 4000	Ask the instructor to recreate the topic (delete + create) and publish once
Counts spread evenly with no hot partition	parse.key missing — whole line used as value	print.key shows null	Recreate the topic; publish with both --property options
Partition read does not exit	--max-messages larger than the partition	kafka-get-offsets for partition 1	Use the exact count (1642) or add --timeout-ms 10000
fulfilment group shows fewer than 3 members	A consumer was not started or exited	--describe --members	Start the missing consumer; wait for the rebalance
awk prints skew=inf or nothing	Command split incorrectly across lines	Run kafka-get-offsets alone	Retype the awk part on one line

General - RPS Data

7. INSTRUCTOR DEBRIEF — 8 MINUTES

7.1 Discussion Questions

Discussion 1 — Diagnosing Partition Skew

Without looking at notes: list the three measurements that proved skew (offsets, keys, assignment).

Why did tripling the consumers not reduce STORE-001's lag?

Key answer to explore: one key → one partition → one consumer; parallelism is capped by key distribution.

Discussion 2 — Fixing Skew

Compare salting, a finer-grained key (order ID) and a dedicated topic for the flagship store.

Which preserves the most ordering, and which balances best?

Key insight: balance and ordering pull in opposite directions — choose the key from the ordering requirement first.

Discussion 3 — Cloud Streaming Alternatives

Which services ask you to plan capacity (shards, TUs, partitions) and which do not?

Would the hot-key problem disappear on Kinesis or Event Hubs? On Pub/Sub?

Key answer: shards and partitions have the same hot-key limit; Pub/Sub has no partitions but ordering keys still serialise one key's messages.

Discussion 4 — Production Considerations

What would you monitor to catch skew before customers notice?

Answer: per-partition bytes-in and log-end offsets, per-partition lag, and a skew ratio alert (for example above 2.0).

7.2 Common Mistakes — Learn From These

General - RPS Data

Mistake	Why It Happens	Consequence	Prevention
Adding consumers to fix a hot partition	It works for even load	No improvement; wasted resources	Measure skew first
Adding partitions to fix a hot key	More partitions = more parallelism	Hot key just moves; ordering broken	Change or salt the key
Salting without checking ordering needs	Balance looks better	Out-of-order processing per entity	Agree ordering rules first
Choosing a cloud service by name recognition	Familiar vendor	Quota or retention limits hit later	Size with published limits
Ignoring egress when sizing	Only ingress is quoted	Consumers throttled	Size for total read throughput too

8. EXTENSION TASKS (Advanced Learners)

Extension A — Choose the Best Salt Count

Use murmur2.py to compute the skew ratio of the salted data for 8, 16, 24 and 32 salts.

Explain why more salts is not always better (hint: hash collisions with 6 partitions).

Key question: how would the result change on 12 partitions?

Extension B — Event Hubs as a Kafka Endpoint

Research how a Kafka client connects to Azure Event Hubs (bootstrap server on port 9093, SASL authentication).

List which Kafka features are and are not supported by the Event Hubs Kafka endpoint.

Explain how this could be used for a phased migration.

Extension C — Managed Kafka Options

Compare self-managed Kafka with Amazon MSK, Confluent Cloud and Azure HDInsight/Event Hubs for Kafka workloads.

List what the operations team still owns in each option.

Present a one-slide recommendation for RetailCo.

General - RPS Data

9. LAB COMPLETION & SIGN-OFF

9.1 Submission Checklist

✓	Completion Criterion	Status
☐	Skewed orders published; counts 63/1642/106/21/42/126 and skew 4.93 recorded	☐ Pass / ☐ Fail
☐	Hot key STORE-001 and the overloaded consumer (1,705 records) identified	☐ Pass / ☐ Fail
☐	Salted orders published; counts 162/244/304/473/391/426 and skew 1.42 recorded; ordering trade-off explained	☐ Pass / ☐ Fail
☐	Kafka, Kinesis, Event Hubs and Pub/Sub mapped and sized; recommendation justified	☐ Pass / ☐ Fail
☐	All 8 validation checks completed and recorded	☐ Pass / ☐ Fail
☐	All observation/answer boxes in this guide completed	☐ Pass / ☐ Fail

9.2 Grading Criteria

Assessment Area	Criteria	Marks	Weight
Task 1 — Reproduce Skew	Exact counts and skew ratio recorded	15	15%
Task 2 — Diagnose	Hot key and overloaded member evidenced; skew vs slow consumer explained	30	30%
Task 3 — Fix	Salting applied and measured; ordering trade-off explained	20	20%
Task 4 — Cloud Comparison	Mapping and sizing correct; recommendation justified	25	25%
Validation & Documentation	All 8 validation checks completed; answers complete and accurate	10	10%
TOTAL		100	100%

9.3 Instructor Sign-Off

General - RPS Data

Field	Value
Lab Result	☐ PASS ☐ PARTIAL ☐ REPEAT REQUIRED | Score: _____ / 100
Instructor Name	_____
Instructor Signature & Date	_____
Remarks / Feedback	_____ _____
Completion Date & Time	_____

General - RPS Data

10. QUICK REFERENCE — SKEW & STREAMING PLATFORM CARD

Bookmark this page. Every command you need in SOP order.

Category	Command / Fact	Purpose
Measure	./k kafka-get-offsets.sh --bootstrap-server $BS --topic t	Records per partition
Measure	... | awk -F: '{s+=$3; if ($3>m) m=$3; n++} END {print m/(s/n)}'	Skew ratio
Keys	./k kafka-console-consumer.sh ... --partition p --offset earliest --formatter-property print.key=true	Keys on one partition
Assignment	./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group g	Which member owns which partition
Predict	python3 -c "from murmur2 import partition; print(partition('KEY', n))"	Partition of a key
Fix	key = entity + '#' + (id % N)	Salt a hot key over N sub-keys
Kinesis	Shard: 1 MB/s or 1,000 rec/s in, 2 MB/s out; retention 24 h – 365 days	AWS sizing
Event Hubs	TU: 1 MB/s or 1,000 events/s in, 2 MB/s out; Standard ≤ 32 partitions, 7 days	Azure sizing
Pub/Sub	No partitions; ordering keys; subscription retention 7 days default (max 31)	Google Cloud model
Clean-up	docker compose down -v (end of day only)	Remove the cluster and its data

IT Training Division | NIIT iRUN Framework | Confidential — For Training Use Only | Do Not Distribute